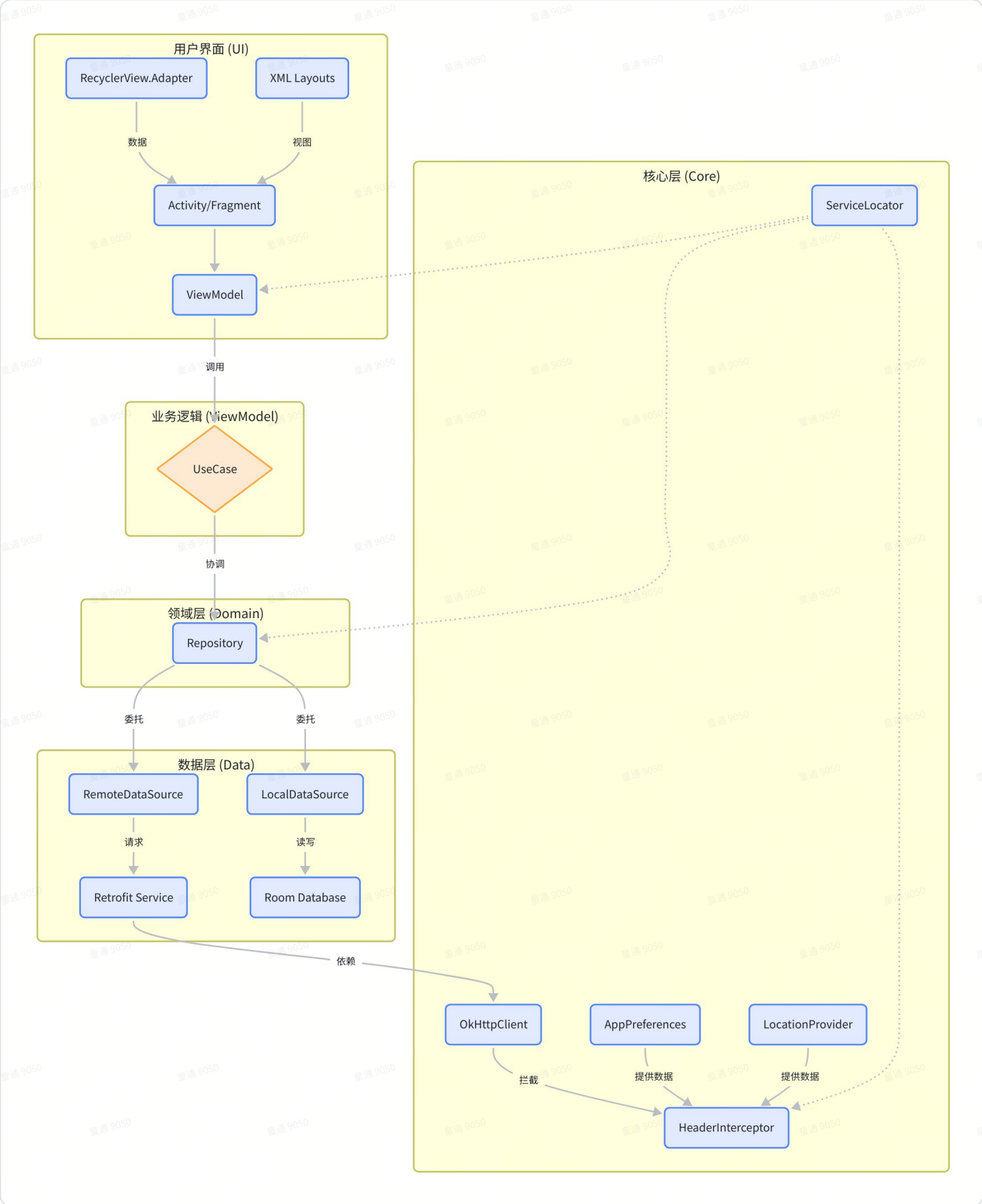
户外轨迹 App - 原生 Android 客户端运行说明

一、架构与目录

1.1 核心架构

本项目采用 MVVM（Model-View-ViewModel）架构模式，并遵循 Google 推荐的 Android 应用架构指南。架构分层清晰，职责明确，便于维护与扩展。



- **UI 层 (`ui`)**: 由 Activity、Fragment、Adapter 和 XML 布局文件组成。负责展示数据和响应用户交互。
- **ViewModel 层 (`viewmodel`)**: 存储和管理与 UI 相关的数据，并处理业务逻辑。通过 `LiveData` 或 `StateFlow` 将数据暴露给 UI 层。

- **Domain 层 (`domain`)**: 可选层，用于封装复杂的业务逻辑或在多个 ViewModel 之间复用的逻辑（以 UseCase 形式存在）。本项目为简化结构，暂未显式创建该层，逻辑直接在 ViewModel 中处理。
- **Data 层 (`data`)**: 包含 Repository、DataSource、Retrofit Service 和数据模型（Model）。负责从网络或本地（数据库、缓存）获取数据。
  - **Repository**: 作为单一数据源入口，对上层屏蔽数据来源的细节。
  - **DataSource**: 区分远程（Remote）与本地（Local）数据源。
- **Core 层 (`core`)**: 包含应用范围内的核心组件，如网络配置、依赖注入（ServiceLocator）、SharedPreferences 封装、定位服务等。

1.2 目录结构

代码块

```
android-client-kotlin/
├── app/
│   ├── src/
│   │   ├── main/
│   │   │   ├── java/com/example/outdoortrack/
│   │   │   │   ├── core/          # 核心组件 (网络、定位、配置、DI)
│   │   │   │   ├── data/          # 数据层 (Repository, API, Model, DB)
│   │   │   │   ├── ui/            # UI 层 (Activity, Fragment, ViewModel, Adapter)
│   │   │   │   └── OutdoorTrackApplication.kt # Application 类
│   │   │   ├── res/
│   │   │   │   ├── layout/        # XML 布局文件
│   │   │   │   ├── values/        # 资源文件 (strings, colors, themes)
│   │   │   │   └── navigation/    # Navigation Component 路由图
│   │   │   └── AndroidManifest.xml
│   │   └── test/                  # 单元测试
│   ├── build.gradle.kts           # App 模块 Gradle 配置
│   └── proguard-rules.pro
├── build.gradle.kts               # 项目级 Gradle 配置
├── gradle.properties
└── settings.gradle.kts

```

二、依赖与配置

2.1 主要依赖库

- **Kotlin + Coroutines**: 异步编程基础。

- **Jetpack Lifecycle**: `ViewModel`, `LiveData` 用于实现 MVVM 架构。
- **Jetpack Navigation**: 单 Activity + 多 Fragment 架构的路由管理。
- **Retrofit & OkHttp**: 强大的 HTTP 客户端，用于网络请求。
- **Glide**: 高效的图片加载库。
- **Material Components**: 提供丰富的 Material Design 风格 UI 组件。
- **Room**: 本地数据库，用于持久化存储轨迹采样点。
- **AMap SDK**: 高德地图与定位服务（占位集成）。

2.2 环境配置

在根目录下的 `local.properties` 文件中配置以下变量：

- `API_BASE_URL` : 后端服务的 URL 地址。
- `AMAP_API_KEY` : 高德地图 SDK 的 API Key。
- `X_CLIENT_VERSION` : 客户端版本号。

这些值将通过 `BuildConfig` 注入到应用中，避免硬编码。

三、头拦截器约定

所有对服务端的 API 请求都会通过一个 OkHttp 拦截器 (`ClientHeadersInterceptor`) 统一添加以下五个 Header：

- **`X-User-Id`** : 当前登录用户的 ID，从 `AppPreferences` 获取。
- **`X-Client-Type`** : 固定为 "android"。
- **`X-Client-Version`** : 应用版本号，从 `BuildConfig` 获取。
- **`X-Client-Lang`** : 客户端语言，从 `AppPreferences` 或系统默认语言获取。
- **`X-Geo`** : "纬度,经度" 格式的地理位置字符串，从 `LocationProvider` 获取。如果定位不可用，则发送占位符 "0,0"。

四、页面与接口映射

页面	主要接口 (Method & Path)	描述
登录页	-	手机号登录，微信登录占位
首页	`GET /api/track/recommend/list`	展示推荐轨迹
首页-正在记录	`GET /api/track/running?user_id=xxx`	检测未结束轨迹

开始记录	`POST /api/track/create`	创建新轨迹
轨迹搜索	`GET /api/track/search/list`	搜索轨迹
正在记录详情	`GET /api/track/{id}/map` `GET /api/track/{id}/detail`	展示实时地图与信息
轨迹记录结束总结	同上	展示地图与汇总信息
上传到云端	`POST /api/track/{id}/upload_cloud`	上传轨迹
他人轨迹详情	`GET /api/track/{id}/map` `.../summary` `.../collect`	展示他人轨迹、收藏状态
个人中心	`GET /api/user/{uid}/detail`	展示用户信息
设置	`PUT /api/user/profile/*`	修改用户信息

五、运行步骤

1. **环境准备**: 确保 Android Studio 和 JDK 17+ 已正确安装。
2. **打开项目**: 在 Android Studio 中打开 `android-client-kotlin/` 目录。
3. **配置**: 复制 `local.properties.example` 为 `local.properties` 并填入真实值。
4. **同步 Gradle**: 等待项目依赖下载完毕。
5. **运行**: 选择一个设备或模拟器，点击运行按钮。

六、注意事项与后续增强

- **占位实现**: 高德地图、微信登录、相机/相册选择等功能均为占位实现，需要进一步集成相应 SDK。
- **错误处理**: 当前错误处理较为简单（如 `Toast`），生产环境应设计更完善的错误反馈机制。
- **轨迹算法**: 轨迹采样、滤波、纠偏等高级功能暂未实现，当前仅记录原始定位点。
- **UI/UX**: 界面为基础实现，可根据 Figma 设计稿进一步优化样式和交互细节。